BeaCIM: A Digital Compute-in-Memory DNN Processor With Bi-Directional Exponent Alignment for FP8 Training

Yang Zhou, Yang Wang, Wenxin Yin, Yazheng Jiang, Jingchuan Wei, Yubin Qin, Yang Hu, *Member, IEEE*, Shaojun Wei, *Fellow, IEEE*, and Shouyi Yin, *Fellow, IEEE*

***Abstract*—Previous digital Compute-In-Memory (DCIM) exhibits limitations in Deep Neural Network (DNN) training with FP8, which is playing an increasingly important role in model training. Most of the previous DCIM purely align exponents to the maximum one when computing floating-point data. This method struggles to balance accuracy and energy efficiency when using FP8. In this brief, we propose BeaCIM, a DCIM processor with bi-directional exponent alignment. There are several contributions. First, we propose a new exponent alignment mechanism, which can dynamically adjust the shared exponent (E^*) towards the numerical distribution center of exponents. Second, we use the Shift-on-Product (SOP) method to address the limitation of data bitwidth, and present E^* calculator based on the Ordinary Least Squares (OLS). Finally, we performed RTL-level circuit implementation and evaluated BeaCIM using various datasets and models. Our experiments show the following results: (1) Compared with the standard hybrid FP8 training, our bi-directional exponent alignment FP8 training exhibits an average top-1 accuracy drop less than 0.75% across different models and datasets. (2) BeaCIM can achieve an energy efficiency of 31.5 TFLOPS/W at FP8, which is 2.6-19.3× better than state-of-the-art works.**

***Index Terms*—Compute-in-memory, FP8, DNN training.**

I. Introduction

TRADITIONAL digital deep neural network (DNN) processors are facing challenges posed by the memory wall. Previous efforts have focused on optimizations from both algorithmic and hardware perspectives. On the algorithmic side, training DNN with FP8 shows notable advantages because of its shorter bitwidth. From a hardware perspective, digital Compute-in-memory (DCIM) is one of the methods that can break through the memory wall [1], [2], [3], [4], [5], [6]. However, currently, these two approaches are not well-coordinated. Because most of the previous DCIM can not effectively support DNN training with FP8.

Currently, there are two main methods for floating-point (FP) calculating in DCIM: quantization and exponent pre-alignment. Conventional FP units cannot be directly implemented in DCIM. Because implementing FP MAC in DCIM complicates the in-memory computational logic and ultimately leads to a decrease in storage density. According to [7], an FP16 multiply–and-accumulate (MAC) operation will cost 2.15× more area overhead than an INT16 MAC. Purely quantifying FP data into fixed-point data is not a viable solution either. Because fixed-point data have a much smaller dynamic range, which is crucial of backpropagation [8]. Exponent pre-alignment is widely used in previous DCIM designs [5], [6]. The exponents of FP data in a same group will be aligned to the maximum exponent (E_{max}). Then their corresponding mantissas can be computed with INT MAC directly.

However, several challenges remain unresolved. The first one is accuracy. Data in DNN model exhibits a long-tailed bell curve [9], [10], with the majority of values concentrating in a small region and a minority of large values extending over a wide range, especially for gradient data. Thus a large outlier can cause significant numerical error, and finally result in accuracy loss. Our experiments have indicated that using traditional FP E_{max} alignment for FP8 training results in very poor accuracy. Secondly, most exponents need to align with the E_{max}, which means most mantissas need to be shifted, leading to increased power consumption.

In this brief, we propose a DCIM processor BeaCIM for FP8 training. First, we propose a bi-directional exponent alignment strategy, which adopts Shift-on-Product (SOP) method to dynamically extend the data bitwidth during partial product computation. It effectively mitigates accuracy loss while avoiding the extra computational cycles. Second, we derive the numerical equation based on the Ordinary Least Squares (OLS) to find the optimal exponent alignment values E^*, which can minimize the total amount of shifting and reduce power

Received 31 July 2024; revised 24 September 2024, 8 November 2024, and 10 January 2025; accepted 5 February 2025. Date of publication 11 February 2025; date of current version 28 March 2025. This work was supported in part by the National Key Laboratory of Human–Machine Hybrid Augmented Intelligence, Xi'an Jiaotong University; in part by NSFC under Grant 62304121, Grant 62125403, Grant U19B2041, and Grant 92164301; in part by the National Key Research and Development Program under Grant 2021ZD0114400; in part by the Shanghai Municipal Science and Technology Major Project; in part by the Beijing National Research Center for Information Science and Technology; and in part by the Beijing Advanced Innovation Center for Integrated Circuits. This brief was recommended by Associate Editor Q. Feng. *(Corresponding authors: Yang Wang; Shouyi Yin.)*

Yang Zhou, Yang Wang, Wenxin Yin, Yazheng Jiang, Jingchuan Wei, Yubin Qin, Yang Hu, and Shaojun Wei are with the School of Integrated Circuits, Beijing Innovation Center for Future Chip, and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China.

Shouyi Yin is with the School of Integrated Circuits, Beijing Innovation Center for Future Chip, and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China, and also with the Shanghai AI Lab and the International Innovation Center, Tsinghua University, Shanghai, 200063, China (e-mail: yinsy@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCSII.2025.3541101

1549-7747 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:31:33 UTC from IEEE Xplore. Restrictions apply.

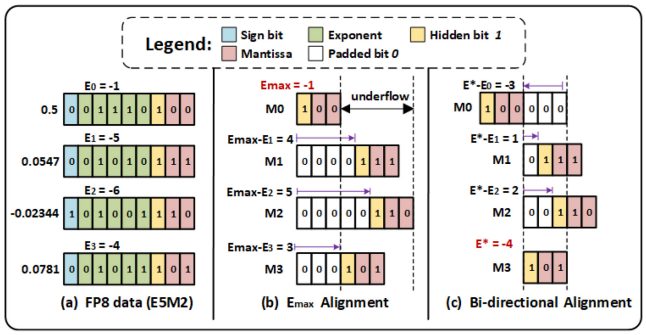


Fig. 1. Different exponents alignment strategy. (a) Original FP8 data. (b) E_{max} alignment. (c) Bi-directional alignment.

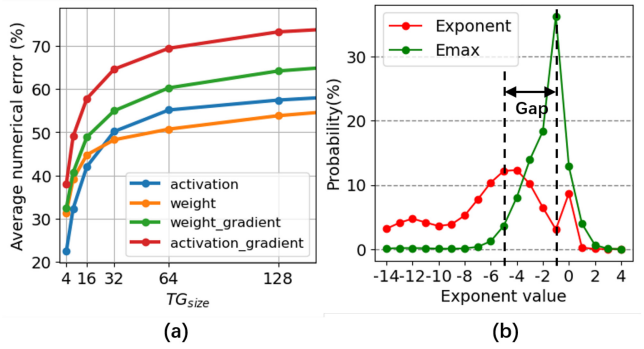


Fig. 2. (a) Average numerical error with different group size. (b) Distribution gap between exponents and E_{max} of IA gradient.

consumption. Compared with standard hybrid FP8 training, our method only incurs an average Top-1 accuracy drop less than 0.75%. Finally, we implement the above techniques on BeaCIM and evaluate the processor with TSMC 28nm process. BeaCIM achieve an energy efficiency of 31.5 TFLOPS/W at FP8, which is 2.6-19.3× better than state-of-the-art works.

II. Background

FP8 has two commonly used data formats: 1-4-3 and 1-5-2 (sign-exponent-mantissa), referred to as E4M3 and E5M2 [11], [12]. Existing DCIM architecture lacks efficient support for FP8 computation. Quantization FP8 to fixed-point data (INT) is the simplest method [3], [10], but the narrow data representation range of INT leads to poor performance during training.

Another E_{max} alignment method can better preserve the numerical range, but it still leads to massive numerical errors on FP8 calculation. We use TG_{size} to describe the amount of FP numbers in the tensor, and all numbers within the tensor share the same exponent. As shown in Fig. 1 (b), find the E_{max} at first. Then, calculate the offset $E_{max} - E$ and right-shift the mantissa based on the offset. Those underflow bits are truncated. This method is susceptible to exponent outliers, which can introduce significant numerical errors.

Simply reducing TG_{size} is insufficient to effectively reduce the numerical errors. Fig. 2 (a) illustrates the curve of numerical errors as TG_{size} changes. The blue and orange lines represent the change curve of input activation (IA) and weight (W) in forward propagation. While the red and green lines represent the change curve of the IA and weights gradients in backward propagation. When the TG_{size} is only 8, the alignment-induced numerical errors in backpropagation exceed 40%. What's more, such small group size is not suitable for DCIM computations. It introduces excessive alignment, increasing hardware overhead. To illustrate the origin of this error, Fig. 2 (b) depicts the distribution of E_{max} and exponents corresponding to IA gradients of ResNet18 at FP8 ($TG_{size} = 128$). There is a significant gap between the numerical distribution center of E_{max} and exponents. Once the gap exceeds the bitwidth of the mantissa, the shifting of mantissa can lead to significant numerical errors.

III. Bi-Directional Exponent Alignment

A. Motivations

One approach to decrease the numerical error is to directly extend the mantissa bitwidth, but this is not cost-effective. Once the bitwidth of mantissa is expended, the amount of cycles required for calculation using the bit-serial approach increases linearly, significantly diminishing energy efficiency. Considering this, we design Shift-on-Product (SOP) mechanism that uses dynamic real-time shifting to avoid directly extending storage bitwidth. The key idea is to move bitwidth extension after multiplication to avoid directly using long bitwidth data for bit-serial calculation.

What's more, We observe that E_{max} occupies a relatively small proportion overall exponents, aligning all exponents to E_{max} results in more mantissa shifts, thereby leading to increased dynamic power consumption. We propose an innovative alignment strategy that reduces the overall amount of shifting by aligning exponents to E^*, which is closer to the numerical distribution center. In Fig. 1, we present an example to illustrate the process. In Fig. 1 (b), all exponents are aligned to the maximum one E_0, the total amount of shifting is 12. If we set the alignment target value to E_3 (-4), the bit shifting amounts can be reduced to 6, as illustrated in Fig. 1 (c). Therefore, the key idea is to identify an optimal value for E^* that minimizes the total shifting of mantissa bits.

In summary, the bi-directional exponent alignment comprises two parts: dynamic bitwidth extension via SOP to minimize accuracy loss, and E^* alignment to reduce shifting and dynamic power consumption.

B. E^* Calculator

We propose an efficient method based on the Ordinary Least Squares to find the optimal E^*. Our optimization goal is to minimize the total amount of mantissa shifting. This is an optimization problem regarding E^*, where the difference between E^* and other exponents E_i can be measured by the square of the Euclidean distance. Considering the following equation:

$$\text{minimize}: \sum_{i=0}^{N-1} \left(E^* - E_i\right)^2 \qquad E^* \in \mathbb{Z} \tag{1}$$

By calculating the zero points of its partial derivative with respect to E^*, we can determine that the optimal choice is the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:31:33 UTC from IEEE Xplore. Restrictions apply.

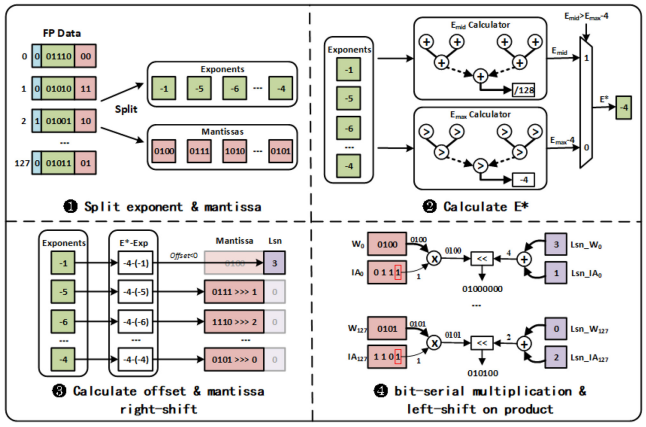


Fig. 3. The 4 steps of Shift-on-Product mechanism.

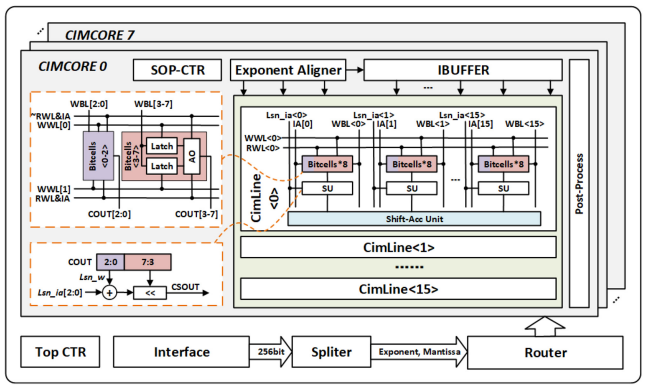


Fig. 4. Top-level of DCIM architecture.

mean of exponents (E_{mid}). Since this midpoint may not be an integer, we truncate its decimal part. While this may yield a suboptimal solution, it is acceptable in terms of hardware overhead. It should be noticed that, the farther the distance between E^* and E_{max}, the larger the shifter needed in the SU. To balance precision and hardware overhead, we limit the maximum distance threshold between E^* and E_{max} ($Dist_E$) to 4, which is enough according to our experiments (more details in Section V-B). The calculation formula of E^* is shown in equation (2). In weight and IA pre-alignment, $E^* = E_{mid}$ is the majority case, while in gradient pre-alignment, $E^* = E_{max} - 4$ is dominant due to the wide distribution of gradient.

$$E^* = \mathbf{max}\left(E_{max} - 4, \lfloor \frac{1}{N}\sum_{i=0}^{N-1} E_i \rfloor\right) \tag{2}$$

C. Shift-on-Product Mechanism

We decompose the whole process into 4 steps, as shown in Fig. 3. ❶ All 128 FP data are spilt into exponents and mantissas. We combine the sign-bit, hidden "1", original mantissa together, and then convert it into 2's complement. As the format in this example is E5M2, their storage mantissas are 4-bits. ❷ We use equation (2) to calculate E^*. E_{mid} and E_{max} calculators are implemented. E_{mid} calculator is composed of a 128-input adder tree. E_{max} calculator is composed of a 128-input comparator tree. Although the input parallelism is high, the critical path delay is acceptable due to the small input bitwidth (5bit). If $E_{mid} > E_{max} - 4$, E^* is sourced from E_{mid} calculator, otherwise it comes from E_{max} calculator. In this example, we assume the final $E^* = -4$. ❸ We calculate the offset between each exponent and E^*, shifting mantissa according to the offset. Such as the second element, its exponent is -5, so its mantissa (0111) will be right shifted to (0011). For those data with exponents larger than E^*, their mantissas are not shifted and offsets are stored as *Lsn* for left-shifting in the next step. Such as the first element, whose offset is +3. *Lsn* is stored together with the corresponding mantissa. ❹ The partial products of IA mantissas and W mantissas are calculated in bit-serial pattern. Then the partial product is left-shifted according to its corresponding $Lsn_{IA} + Lsn_W$. The multiplication of IA and W can be written as equation (3). For IA_0 and W_0, the current partial product is 0100, their *Lsn* is 3 and 1, thus the final partial product is 0100 << $(3+1) = 01000000$. This product is sent to shift-accumulator for bit-serial accumulation. If the tensor size exceeds TG_{size}, it is partitioned into smaller tiles for individual computation, with the results subsequently aggregated in the post-processing stage.

$$IA \times W = \sum_{i=0}^{N-1}((IA[i] \times W) << (Lsn_{IA} + Lsn_W)) \times 2^i \tag{3}$$

IV. Hardware Architecture

Fig. 4 illustrates the overall architecture of BeaCIM. BeaCIM consists of 8 CIM cores, top-controller, interface, Spliter and data router. The BeaCIM data bus has a bandwidth of 256 bits and interacts with external memory through the interface. Input data is sent to Spliter to split the original FP data into exponents and 2's complement mantissas. After that, exponents and mantissas are sent to each CIM core by the data router based on the configured address. Each CIM core contains an Exponent Aligner, SOP-CTR, Ibuffer, post-process unit and 16× Cimline. Exponent Aligner calculates the optimal E^* and performs mantissa shifting with algorithm introduced in Section III. SOP-CTR is the controller of SOP dataflow. The post-processing unit normalizes the output results of Cimline. Ibuffer integrates 1 Kb local memory for IA mantissa and Lsn_{IA}. Each cycle the ibuffer provides 16 IA data to the cimline in a bit-serial manner, and these 16 bits IA can be shared across all 16 cimlines. In order to support both E5M2 and E4M3 formats, we designed the exponent-related part with 5-bit and the mantissa-related part with 5 bits (sign + hidden "1" + 3 bit mantissa).

Cimline is composed of 16× BU and SU. Each BU contains 8 bitcells, capable of storing 2 rows × 8 bits. SU shifts the product of BU, as described in Section III-C. We implement a latch based 2-row structure proposed by TSMC [3] as the basic bitcell, which has been proven can achieve higher energy efficiency. The bitcell consists of 2 latch and 1 4-inputs AO (AND-OR gate). This structure can be fully implemented by foundary standard cells, thereby enhancing its process reliability. The Latch store weight and the AO is responsible

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:31:33 UTC from IEEE Xplore. Restrictions apply.

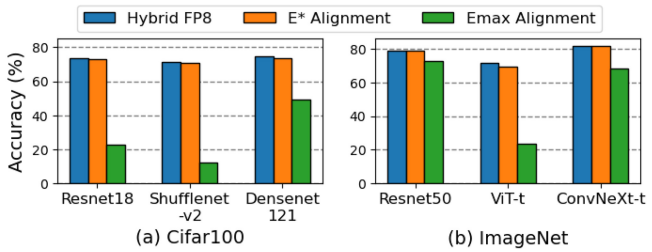
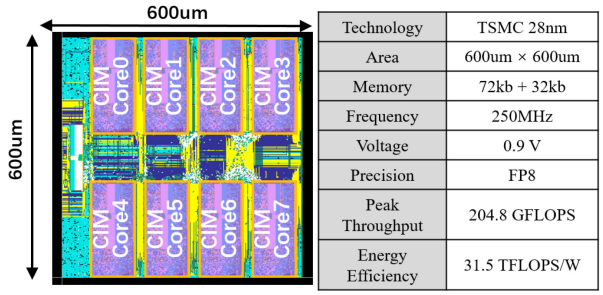


Fig. 5. Training accuracy comparison on different models. (a) Training from scratch on Cifar100. (b) Fine-tuning on ImageNet.



Technology	TSMC 28nm
Area	600um × 600um
Memory	72kb + 32kb
Frequency	250MHz
Voltage	0.9 V
Precision	FP8
Peak Throughput	204.8 GFLOPS
Energy Efficiency	31.5 TFLOPS/W

Fig. 6. Post-layout and performance summary of BeaCIM.

for row selection and bitwise multiplication. We use AO rather than OAI in [3], because positive logic is easier to design. The logic function of the bitcell output is shown in equation (4). When RWL is low, the output (COUT) is the product of I and W_0, otherwise it is the product of I and W_1. The 2-row structure can be used as ping-pong storage to cover the data transfer time. For example, when row[1] is selected by RWL for calculation, we can enable WWL[0] to open the latch of row[0], and write new data through WBL.

$$\text{COUT} = \overline{\text{RWL}} \cdot I \cdot W_0 + \text{RWL} \cdot I \cdot W_1 \tag{4}$$

We partition the BU for different storage. The bitcells[2:0] store the Lsn_W, and the remaining bitcells store the corresponding weight mantissa. The input activations in Ibuffer also adopts the same storage format, ensuring synchronization between the *Lsn* and the mantissa. During computation, the COUT is read out from the BU and sent to the SU simultaneously. And the Lsn_{IA} corresponding to current IA is sent to SU at the same time. COUT[7:3] is the partial product and COUT[2:0] is Lsn_W. Subsequently, the partial product is left-shifted by the summation of Lsn_W and Lsn_{IA} and then sent to the shift-accumulator.

V. Experiment

A. Training Accuracy

Hybrid FP8 training [12] results on GPU are set as benchmarks. We evaluate the performance of traditional E_{max} alignment and our E^* alignment with the same training hyper-parameters. All accuracy evaluations in our study were conducted based on $TG_{size} = 128$. Exponent alignment only occurs in layers involving matrix multiplication or convolution during forward and backward propagation.

We trained ResNet18 [13], ShuffleNetV2 [14], DenseNet121 [15] from scratch using Cifar-100 dataset for 50 epochs, and retrained the output layer of pre-trained models Resnet50, ConvNeXt-tiny [16] and ViT-tiny [17] on Imagenet-1k dataset for 5 epochs. The results are shown in Fig. 5. All networks can hardly converge under E_{max} alignment when training from scratch, the three models achieve top-1 accuracy of only 22.9%, 12.1%, and 49.1%, respectively. Even in fine-tuning there is still a significant gap between and E_{max} alignment and benchmarks, with an accuracy drop of 6.22%, 47.93% and 13.53%. The disparity of accuracy degradation can be attributed to the heterogeneous numerical error tolerances of various DNN models. Lightweight models

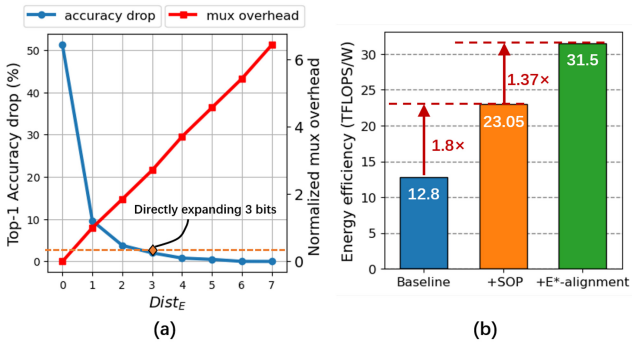


Fig. 7. (a) Training accuracy drop and normalized mux overhead with different $Dist_E$. The training results are derived from ResNet18 on CIFAR-100 dataset. (b) Energy efficiency with techniques breakdown analysis.

are more susceptible to numerical error, thus ShuffleNetV2 exhibit the lowest training accuracy. Besides, some layers in model are highly sensitive to numerical error, which lead to a significant decrease in training accuracy. For instance, the softmax layers in ViT-t.

In contrast, E^* alignment shows very close to benchmark in all cases. E^* alignment can achieve top-1 accuracy of 72.94%, 70.91%, and 73.8% in training from scratch cases, with only 0.65% accuracy drop in average. Similarly, E^* alignment can achieve 78.81%, 69.53% and 81.6% in fine-tuning cases, with only 0.76% accuracy drop in average.

B. Hardware Performance

We synthesized BeaCIM using Synopsys Design Compiler in TSMC 28 nm CMOS technology and accomplished the back-end physical design through Synopsys IC Compiler II. Then we use Synopsys PrimeTime to derive precise power consumption data during computation process.

Fig. 6 depicts the post-layout of BeaCIM IP, with an area of 600um × 600um, and includes memory size of 72kb (ibuffer) + 32kb (bitcells). The BeaCIM IP is simulated with working frequency of 250MHz, achieving peak energy efficiency of 31.5 TFLOPS/W.

We evaluated the impact of different thresholds $Dist_E$ on the convergence accuracy of DNN model. And we use the area of a 2-1 multiplexer (mux) as baseline to quantify the mux overhead introduced in SU for various $Dist_E$. Fig. 7 (a) shows when $Dist_E$ is larger than 4, the accuracy gain is quite limited, while the mux overhead still increases linearly. Thus we set $Dist_E$ as 4 in BeaCIM. Also, we listed the accuracy drop for

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:31:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE COMPARISON

	VLSI'21 [18]	ESSCIRC'22 [19]	ESSCIRC'23 [20]	This work
Technology (nm)	28	28	28	**28**
Area (mm^2)	5.832	1.0	1.56	**0.36**[1]
Precision	BF16	FP8-FP32	FP8	**FP8**
Energy Efficiency (TFLOPS/W)	1.43-13.7[2]	1.64(FP8)	12.1	**31.5**

[1] Only the IP area of BeaCIM, without IO PAD.
[2] Activation sparsity from 0%-90%.

directly extending 3 bits as a comparison. The main difference between these two has been discussed in Section III.

Fig. 7 (b) demonstrates a technique breakdown analysis to clearly illustrate how energy efficiency is improved. The baseline is evaluated with directly expending bitwidth by 4 bits. By employing SOP to dynamically extend bitwidth during calculation, we avoid the additional working cycles, resulting in a 1.8× improvement in energy efficiency. Furthermore, by utilizing E^* as an alignment point, we reduced the amount of mantissa shift, thereby decreasing dynamic power consumption, which led to a 1.37× improvement in energy efficiency.

C. Performance Comparison

In Table I, we compare our design with the state-of-the-art works. Reference [18] is a FP DNN training processor with heterogenous BF16 computing architecture. It calculates exponent using CIM array and uses traditional digital logic for mantissa operations. Reference [19] is a digital CIM macro, which utilizes E_{max} alignment of input-weight pairs. Reference [20] is a digital CIM which supports FP8 calculation with E5M2 format. Reference [18] can support BF16 computations, but it cannot simultaneously accommodate FP8 computations. Reference [19] can support a variety of different FP formats, but the adopted E_{max} alignment in the case of FP8 can lead to a loss of training accuracy. Reference [20] directly employs FP16 adders in the accumulation, which can result in performance degradation. BeaCIM's peak FP energy efficiency is 31.5 TFLOPS/W at FP8, which is 19.2×, 2.6× higher than [19], [20].

VI. CONCLUSION

Previous DCIM with E_{max} alignment for FP calculations suffers from training accuracy loss in FP8. We propose a bi-directional exponent alignment strategy, which can minimize the shift amount of mantissa. We use SOP to address the limitation of data bitwidth and calculate the optimal E^* based on the OLS. The BeaCIM can achieve an average accuracy drop less than 0.75% compared with standard hybrid FP8 training, and obtain the peak energy efficiency of 31.5 TFLOPS/W.

REFERENCES

[1] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2021, pp. 252–254.

[2] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2022, pp. 1–3.

[3] H. Mori et al., "A 4nm 6163-TOPS/W/b 4790-$TOPS/mm^2/b$ SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 132–134.

[4] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[5] F. Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[6] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6t-SRAM computing-in-memory macro for floating-point CNNs," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 128–130.

[7] B. Dally, "Hardware for deep learning," in *Proc. IEEE Hot Chips 35 Symp. (HCS)*, Aug. 2023, pp. 1–58.

[8] X. Sun et al., "Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 32. Red Hook, NY, USA: Curran Assoc., Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/65fc9fb4897a89789352e211ca2d398f-Abstract.html

[9] J. Bai, S. Sun, and W. Kang, "Exploring bit-level sparsity for partial sum quantization in computing-in-memory accelerator," in *Proc. IEEE 12th Non-Volatile Memory Syst. Appl. Symp. (NVMSA)*, Aug. 2023, pp. 32–37.

[10] J. Yue et al., "A 28nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 1–3.

[11] N. Mellempudi, S. Srinivasan, D. Das, and B. Kaul, "Mixed precision training with 8-bit floating point," 2019, *arXiv:1905.12334*.

[12] P. Micikevicius et al., "FP8 formats for deep learning," 2022, *arXiv:2209.05433*.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015, *arXiv:1512.03385*.

[14] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," 2018, *arXiv:1807.11164*.

[15] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," 2018, *arXiv:1608.06993*.

[16] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," 2022, *arXiv:2201.03545*.

[17] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2021, *arXiv:2010.11929*.

[18] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Technol.*, Jun. 2021, pp. 1–2.

[19] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, 2022, pp. 145–148.

[20] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28nm all-digital configurable floating-point in-memory computing macro," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, 2023, pp. 405–408.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:31:33 UTC from IEEE Xplore. Restrictions apply.